Kinematic Modelling and MATLAB-Based Analysis of Delta Robot Algorithms

Ezekiel, Olomiye Moskovitch, Yosef
Nastase, Radu Emilian Oxley, John Raj, Shubham

1 Introduction

Robot manipulators are widely used across industrial, medical and educational domains because of their precision, repeatability and programmability [8]. Common applications include pick-and-place operations, assembly tasks, welding, painting, and laboratory automation. In educational environments, robotic platforms are used to teach fundamentals of kinematics, motion planning, robotics programming, control, and integration [1].

The Delta Robot Kit is a desktop serial manipulator consisting of four revolute joints (1 DOF), two end-effector joints (2 DOF) and one actuated base. It operates in both joint and Cartesian coordinate systems, enabling flexible motion control and trajectory planning. The robot supports multiple motion modes [9] such as point-to-point (PTP), linear (MOVL), and arc-based trajectories, making it suitable for both teaching and experimental robotics tasks.

This is a classical robot, however, the Robot Magician Lite can be categorized as a serial articulated robot arm. Compared to parallel manipulators, serial manipulators offer a larger workspace and greater flexibility, but may suffer from lower stiffness and accumulated positioning errors. Compared to SCARA robots, the Delta robot provides greater vertical flexibility but may have reduced speed and precision in planar tasks.

The Delta Robot Kit is particularly suitable for educational use because it supports both graphical and text-based programmed desktop form [9]. However, limitations include restricted payload capacity (250 g), limited reach (340 mm), and lower industrial grade precision compared to high-end robotic systems [3]. The aim of this project was to develop an analytical and MATLAB-based model of the Delta Robot Kit Lite and to evaluate its forward kinematics, inverse kinematics, Jacobian, singularities and calibration behaviour.

List of Symbols

- θ_i – base rotation angle
- θ_2 – shoulder joint angle
- θ_3 – elbow joint angle
- θ_4 – end-effector rotation angle
- L_1, L_2, L_3 – main arm links lengths

2 Kinematic Modelling

2.1 Robot configuration and assumptions

The kinematic model therefore treats the robot as a 6R serial manipulator, defined by 6 actuated revolute joints, of which only the first four are relevant to controlling tool orientation.

Although the physical robot includes a parallelogram mechanism [4] that helps maintain end-effector orientation, the kinematic model used in this work represents that behaviour through an equivalent serial kinematic representation [6]. This passive element is not treated as an additional actuated degree of freedom, but is used as a modeling device and to preserve the geometric constraints.

Figure 1: Appearance of the Delta Magician Lite showing the physical arrangement of the four revolute joints used in the kinematic model. Adapted from [4]

2.2 Denavit–Hartenberg parameterization

Figure 2: Denavit–Hartenberg frame assignment used for the kinematic modelling of the Delta Magician Lite manipulator

The Denavit–Hartenberg parameters table derived from the frame assignment is shown in Table 1.

Table 1: Denavit–Hartenberg parameter table derived from the frame assignment

Link	α_i (rad)	a_i (mm)	d_i (mm)	θ_i
1 – Base	$-\frac{\pi}{2}$	90	150	θ_1
2 – Rear Arm	0	0	$-L_1$	θ_2
3 – Rear Arm	150	0	0	θ_3
4 – Parallelogram link	0	0	$-L_2$	$-\theta_2 - \theta_3$
5 – End-effector	0	0	$-L_3$	θ_4

The coordinate frames were assigned according to the standard Denavit–Hartenberg convention [6]. Each z-axis was aligned with the axis of rotation of the corresponding joint, while each x-axis was defined along the common normal between adjacent joint axes. The physical parallelogram linkage provides both the orientation of the end-effector and the kinematic constraint that keeps the end-effector orientation approximately constant. This passive linkage is not an independent joint and does not increase the number of actuated degrees of freedom [2].

The Denavit–Hartenberg convention provides a systematic framework for assigning frame transformations and deriving homogeneous transformations for serial manipulators [2, 3].

2.3 Forward kinematics

The forward kinematics for the Delta Magician Lite were derived using the Denavit–Hartenberg convention. A homogeneous transformation matrix was assigned to each joint in order to describe the position and orientation of the link frame relative to the preceding frame. The overall transformation from the base to the end-effector was then obtained by multiplying the individual link transformations [2, 3].

For the equivalent kinematic model used in this project, the total transformation is written as

$$T_{04} = A_1 A_2 A_3 A_4 A_5$$

where A_5 denotes the passive transformation introduced to represent the parallelogram constraint in the physical robot. This passive term is included only to preserve the geometric behaviour of the end-effector and does not introduce an additional actuated degree of freedom.

The forward kinematics transformation matrix provides both the orientation of the end-effector and its Cartesian position relative to the base. The position vector is extracted from the last column of the matrix to derive the inverse kinematics equations and to compare the kinematic model with MATLAB and simulation outputs.

TODO: This is wrong as we need L1 which is non-zero

$$T_{03} = \begin{bmatrix} \cos\theta_1 \cdot \theta_2 & 0 & \sin\theta_1 \cdot \theta_2 & \cos\theta_1 [L_2(\cos\theta_2 + L_3 \cos\theta_2 + \theta_3)] \\ \sin\theta_1 \cdot \theta_2 & 0 & -\cos\theta_1 \cdot \theta_2 & \sin\theta_1 [L_2(\cos\theta_2 + L_3 \cos\theta_2 + \theta_3)] \\ 0 & 1 & 0 & h + L_2 \sin(\theta_2) + L_3 \sin(\theta_2 + \theta_3) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where, θ_1 is the base rotation, θ_2 is the shoulder joint angle, θ_3 is the elbow joint angle, θ_4 is the tool rotation, L_2 is the rear arm length, L_3 is the forearm length, h is the base height. Note that L_1 is not included in the kinematics because it is a constant term that does not affect the velocity mapping.

Analytical Jacobian

The analytical Jacobian satisfies $\dot{x} = J_a(q)\dot{q}$ and is given by

$$J_a(q) = \begin{bmatrix} -\sin\theta_1 \cdot \cdots & \cos\theta_1[-L_2 \cdot \sin(\theta_2) - L_3 \cdot \sin(\theta_2 + \theta_3)] & -L_3 \cdot \sin(\theta_2 + \theta_3) \cdot \cos\theta \\ \cos\theta_1 \cdot \cdots & \sin\theta_1[-L_2 \cdot \sin(\theta_2) - L_3 \cdot \sin(\theta_2 + \theta_3)] & -L_3 \cdot \sin(\theta_2 + \theta_3) \cdot \sin\theta \\ 0 & L_2 \cdot \cos(\theta_2) + L_3 \cdot \cos(\theta_2 + \theta_3) & L_3 \cdot \cos(\theta_2 + \theta_3) \\ 0 & \sin(\theta_1) & \sin(\theta_1) \end{bmatrix}$$

The base offset h appears in the forward kinematics of z, but it does not appear in the Jacobian because it is a constant term. Its derivative with respect to all joint variables is zero.

Singularities

A singularity occurs when the determinant of the analytical Jacobian becomes zero:

$$\det(J) = L_3 L_2 \sin(\theta_3) = 0$$

This expression shows that singularities occur when either $\sin(\theta_3) = 0$ or $r = 0$. The condition $\sin(\theta_3) = 0$ corresponds to the arm being fully stretched or fully folded, so that the links become collinear. The condition $r = 0$ corresponds to the wrist lying on the base axis, where the manipulator loses motion authority in certain horizontal directions. Near these configurations, the velocity mapping becomes ill-conditioned and large joint velocities may be required to generate small end-effector motions [5].

2.5 Inverse kinematics

The inverse kinematic problem was solved using a geometric approach, because it provides closed-form solutions and is well suited to two-link planar arm structures [2, 3].

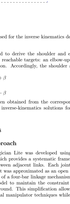

Figure 4: Top-view geometry used to compute the base joint angle θ_1.

The base joint angle was obtained from the projected position in the x–y plane:

$$\theta_1 = \operatorname{atan2}(y, x)$$

The radial distance in the horizontal plane is then $r = \sqrt{x^2 + y^2}$. The remaining problem is reduced to a planar two-link geometry in the r – z plane [2]. A geometric inverse-kinematics method was selected because it provides closed-form solutions and avoids iterative solvers, which are unnecessary for a 3-DOF arm.

Figure 5: Planar geometric construction used for the inverse kinematics derivation.

Figure 5 shows the planar geometry used to derive the shoulder and elbow angles. The construction uses the cosine rule to relate the shoulder and elbow flexion angles, and an elbow-down configuration. Accordingly, the shoulder angle is written as

$$\theta_2 = \phi + \beta$$

for one branch and

$$\theta_2 = \phi - \beta$$

for the other. The elbow angle θ_3 is then obtained from the corresponding trigonometric relation, giving two valid inverse-kinematics solutions for the same target position.

3 Materials and Methods

3.1 Kinematic Modelling Approach

The kinematic model of the Delta Magician Lite was developed using the Denavit–Hartenberg (DH) convention, which provides a systematic framework for describing the spatial relationship between adjacent links. Each joint was modelled as a revolute joint with one rotation about its corresponding z-axis. Static elastic, despite the physical presence of a four-bar linkage mechanism, the passive joints were not treated as independent degrees of freedom in the kinematic model, since they add no extra degrees of freedom. This simplification allows the system to be analysed using standard serial manipulator techniques while preserving key geometric relations.

3.2 Inverse Kinematics Strategy

A geometric approach was selected for solving the inverse kinematics problem. This method was chosen due to the relatively simple structure of the robot and the availability of clear geometric relationships between joint angles and end-effector position.

The inverse kinematics solution was derived by:

- Projecting the target position onto the base axis using $\theta_1 = \operatorname{atan2}(y, x)$
- Reducing the 3D problem into a 2D planar problem in the (r, z) plane
- Applying trigonometric relationships and cosine law to determine θ_2 and θ_3
- Considering both elbow-up and elbow-down configurations

This approach was preferred over numerical methods due to its computational efficiency, analytical solutions and suitability for real-time applications.

3.3 Dynamic modelling approach

The dynamic behaviour of the manipulator was considered using the standard rigid-body robot dynamics formulation [5]:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

where τ is the vector of joint torques, $M(q)$ is the inertia matrix, $C(q, \dot{q})$ represents Coriolis and centrifugal effects, and $G(q)$ represents gravity effects.

In the present project, the dynamic equations were incorporated at a conceptual level and modelling tool was used to verify simplicity therefore the MATLAB. However, a complete high-fidelity dynamic implementation was limited by the lack of reliable mass, inertia, friction and actuator parameters. Instead, the focus was on kinematics, Jacobian analysis and simulation validation. Future work could focus on establishing the dynamic form of the equations and explaining how such terms would be integrated into the MATLAB model for future extension.

3.4 MATLAB Implementation

The robot model was implemented in MATLAB using a modular approach. Individual functions were created for the Denavit–Hartenberg transformation, forward kinematics evaluation, inverse kinematics solution, Jacobian computation, and workspace visualization. Symbolic computation was used where appropriate to derive compact expressions for the forward kinematics and Jacobian, while numerical functions were used for plotting and simulation.

The implementation was designed so that a simple Cartesian motion could be provided as input, the corresponding joint angles could be computed through inverse kinematics, and the resulting configuration could be visualized through forward kinematics reconstruction. Additional checks were included to detect unreachable target and singular configurations.

3.5 Testing Framework

A structured testing framework was used to validate the correctness of the mathematical model and MATLAB implementation. Forward kinematics was tested by predicting known joint configurations and comparing the resulting end-effector position with the expected value. Inverse kinematics correctness was validated by solving for joint variables from a target Cartesian position and checking whether the forward kinematics reproduced the original position. Jacobian tests were used to verify singularity detection and warn when operation too unreachable targets. Together, these checks ensured internal consistency between the analytical model, the numerical implementation, and simulation.

4 Results and Validation

The MATLAB model was validated using forward-kinematics, inverse-kinematics and singularity test cases. Forward kinematics was checked by comparing computed end-effector positions against known joint configurations, inverse kinematics was verified by reconstructing joint configurations from target positions, Jacobian tests were used to confirm singularity detection and rejection of unreachable targets.

Table 2: Validation of forward and inverse kinematics test results table

Test case	Input	Expected	Result
Forward kinematics	$\theta_1 = 0.785$ rad, $\theta_2 = 0.785$ rad, $\theta_3 = 0.785$ rad	$(x, y, z) = (0.9437, 0.106$	
Inverse kinematics	Target $(0.1837, 0.1061, 0.291)$ m	$\theta_1 = 0.57236, 0.7854, 0.0$	
Jacobian	$\theta_2 = 0$, $\theta_3 = 0$	$\det(J) = 0$	
Singularity test	Multiple valid solutions	11/11 tests passed	

The validation results show good agreement between the analytical model and the MATLAB implementation. In particular, the consistency between the forward kinematics output and the inverse kinematics reconstruction shows that the position mapping is internally correct. The singularity and unreachable target tests further confirm that the implementation correctly handles important edge cases within the robot workspace.

Figure 6: Successful inverse kinematics solution using the MATLAB-based UI.

Figure 7: Detection of unreachable target in inverse kinematics.

The system correctly identifies when a target position lies outside the robot's reachable workspace. In this example, the computed distance 4 exceeds the maximum reach of the robot (max = 3.0 m), confirming that no valid solution exists. This demonstrates that the inverse kinematics implementation includes proper workspace validation and prevents physically invalid solutions.

5 Discussion and Conclusion

This project developed and validated a kinematic model of the Delta Magician Lite using the Denavit–Hartenberg convention and implemented the analysis in MATLAB. Forward kinematics, inverse kinematics, Jacobian analysis, and singularity detection were developed and validated, and simulation and visualization were identified from the analytical model. The resulting MATLAB implementation was sufficiently robust to handle valid joint positions and reject targets lying outside the reachable workspace.

One of the main challenges was representing the physical parallelogram mechanism of the robot within a tractable analytical framework. This was addressed by treating the passive linkage as a kinematic constraint rather than as an independent actuated degree of freedom. This approach preserved the correct kinematic constraint without introducing an additional actuated degree of freedom. Another challenge was the derivation of inverse kinematics in a form suitable for reliable computation; this was solved by reducing the problem to a planar geometric construction in the r – z plane. A further limitation concerned dynamics, since accurate link masses, centers of mass and inertia tensors were not available, so a full dynamic model was not implemented numerically.

Despite these limitations, the final model provides a robust analytical and computational framework for studying the motion of the Delta Magician Lite. The implementation is modular, so the model is suitable for forward kinematics, inverse kinematics, singularity detection and validation testing in MATLAB. Future work could extend the model to include trajectory generation, singularity-aware motion planning, actuator constraints, joint limits and closed-loop control in order to improve robustness and practical applicability.

Figure 8: Principal dimensions of the Delta Magician Lite used to support the geometric assumptions of the model. Adapted from [4].

6 References

References

[1] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, Springer, 2nd edition, 2017.

[2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*. Pearson Education, Upper Saddle River, 3rd edition, 2005.

[3] J. Denavit and R. S. Hartenberg, A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22:215–221, 1955.

[4] Elephant Robotics. Delta Magician Lite (Desktop-based). n.d. [Online]. Available: https://mycobot.ai/product-manual/delta-magician-lite/desktop-based/#Introduction. Accessed: 20/Mar/2026.

[5] Elephant Robotics and apply inverse kinematics to robot arm. 2023. [Online]. Available: https://ask.mathworks.com/help/robotics/ug/apply-inverse-kinematics-to-robot-arm.html.

[6] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Springer London, 2009.

[7] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, Hoboken, 2005.